ОТКРЫТЫЙ УРОК · МАТЕРИАЛЫ ПРЕПОДАВАТЕЛЯ

Этапы логического проектирования программы

Проектируем программу бронирования виртуальной лаборатории: от предметной области до правил и тестовых сценариев. Основная практика — проектирование программы. Прежний конструктор VM сохранён как дополнительный пример предметной области.

1. Паспорт занятия

Параметр	Организация
Тема	Этапы логического проектирования программы. Системный подход. Концептуальная модель. Предметная область.
Продолжительность	50 минут, 19 октября 2026 года, 11:10–12:00, ФМФ 503.
Участники	ВТРО-2(3), бакалавриат. Работа индивидуально или парами со сменой ролей.
Подготовка	Умение читать простую схему и сравнивать временные интервалы. Язык программирования, SQL и установка VM для основного задания не требуются.
Результат	Сохранённый проект: цель, сущности, граница системы, связи, алгоритм, ограничения, ожидания тестов и объяснение.

2. Цель и результаты обучения

Научиться переводить задачу из предметной области в согласованную модель данных и поведения программы до реализации.

1. Выделить предметную область, цель и существенные понятия без смешения объектов, интерфейсных действий и оборудования.

2. С позиции системного подхода определить входы, выходы, функции и внешнюю среду.
3. Построить концептуальную модель из трёх сущностей и двух связей 1:N.
4. Составить логический порядок обработки заявки и объяснить ветви отказа.
5. Выразить ограничения временных интервалов и проверить граничные случаи.
6. Объяснить, какую ошибку удалось найти до написания программы.

3. Термины для объяснения

Понятие	Смысл и пример
Предметная область	Часть реального мира, которую описываем: использование лабораторий, студенты и записи бронирований.
Системный подход	Рассмотрение цели, частей, связей, границ и внешней среды. Программа обрабатывает заявку; студент выбирает время, сотрудник ремонтирует оборудование.
Концептуальная модель	Описание основных понятий, связей и правил без выбора языка и СУБД. Студент — бронирование — лаборатория.
Логическое проектирование	Уточнение структуры данных, операций, условий и алгоритмов обработки. Сначала проверки, затем сохранение. Конкретная реализация ещё не выбирается.

> Последовательность «требования → сущности и границы → модель → алгоритм и ограничения → проверка» используется как рабочая структура этого урока. Это не единственная универсальная классификация; проектирование итеративно, новые тесты могут потребовать изменения модели.

4. Задание заказчика

Создать программу, в которой студент выбирает лабораторию и интервал, получает подтверждение или отказ и может отменить запись. Для каждой записи известны студент, лаборатория, начало и конец. В этой практике вход пользователя считаем уже выполненным.

- Конец строго позже начала.
- Активные записи одной лаборатории не пересекаются.

- Конец предыдущей записи может совпадать с началом следующей.
- В разных лабораториях одинаковые интервалы допустимы.
- При ошибке новая запись не сохраняется.

Это учебный логический проект. Реальные бронирования университета не создаются. Квест проверяет проектные артефакты; свободные объяснения сохраняются, но их смысл оценивает преподаватель.

5. Подготовка до урока

1. Открыть пульт, выбрать тип новой группы «Логическое проектирование программы» и создать отдельную группу для урока. Старые группы VM не менять.
2. Проверить ссылку: она должна вести в `/logic/`. Показать QR через «Показать QR для студентов». Ввести секрет до вывода на проектор.
3. Проверить вход на втором устройстве и появление этапов в пульте. Самостоятельный режим без кода хранит проект только в браузере; JSON — самоотчёт.
4. Скачать презентацию и PDF плана. Подготовить бумагу для схем и резервную парную работу при отсутствии сети.
5. Открыть страницы: презентация, проектный квест, пульт, приглашение. Прежний VM-конструктор показывать не более 1 минуты как иллюстрацию лаборатории.

6. Поминутный сценарий — 50 минут

Время	Преподаватель	Студенты и результат
0–3	«Два студента выбирают одну лабораторию в одно время. Почему недостаточно просто нарисовать кнопку “Забронировать”?»	Предлагают правила и возможные ошибки.
3–9	Шесть коротких слайдов: область, система, модель, алгоритм, тесты.	Называют объект, действие, вход и выход системы.
9–12	QR новой группы, проверка участников; роли оператор и аналитик.	Входят по нику; оператор управляет, аналитик объясняет.
12–17	Задаёт вопросы о цели и границах.	Этапы 1–2: формулируют цель, выбирают сущности и внутренние функции.

17–23	Просит объяснить каждую связь на жизненном примере.	Этап 3: строят две связи 1:N. Показывают, к кому относится одна запись.
23–29	«Можно ли сохранять до проверки свободного времени?» Пары меняются ролями.	Этап 4: переставляют шаги алгоритма; объясняют путь отказа.
29–35	Показывает интервалы 10–11, 11–12 и 10:30–11:30.	Этап 5: собирают ограничения длительности, области сравнения и пересечений.
35–41	Просит сначала предсказать результат, потом проверить.	Этап 6: шесть тестов и собственный вывод; сохраняют проект.
41–47	Выбирает 2–3 проекта. Даёт новую ситуацию: одновременно пришли две заявки.	Защищают модель; объясняют необходимость атомарной проверки и сохранения в реальной системе.
47–50	Индивидуальный выходной билет и обратная связь.	Называют предметную область, правило и ошибку, найденную до кода.

7. Методы преподавания

Метод	Применение	Проверка результата
Проблемное обучение	Задача конфликтующих заявок до знакомства с терминами.	Студент объясняет необходимость правил.
Проектное задание	Пары последовательно создают взаимосвязанные артефакты одной программы.	Цель, модель и поведение согласованы.
Визуальное моделирование	Выбор сущностей, построение связей и перестановка блоков алгоритма.	Студент читает схему своими словами.
Предскажи — проверь — объясни	Ожидания для шести сценариев до запуска проверки правил.	Есть причинное объяснение граничного случая.

Парное обучение	Оператор и аналитик меняются после построения модели.	Оба могут защитить решение индивидуально.
Формирующее оценивание	Пульт, вопросы, подсказка сначала вопросом, затем указанием на противоречие.	Помощь по затруднению, а не по месту в рейтинге.
Рефлексия	Вывод об ошибке до программирования и перенос на новый сценарий.	Студент отличает прохождение интерфейса от понимания проекта.

Методы — описание запланированной организации занятия, а не заявление об измеренной эффективности.

8. Ключ преподавателя

Этап	Ожидаемый артефакт	Баллы квеста
1 · Область	Студент, лаборатория, бронирование. Цель своими словами; длина текста проверяется автоматически, смысл — устно.	10
2 · Граница	Внутри: валидация, доступность, хранение, отмена. Вне: выбор времени человеком и ремонт.	15
3 · Модель	Студент 1:N Бронирование; Лаборатория 1:N Бронирование.	20
4 · Алгоритм	Получить → проверить данные → проверить доступность → сохранить → подтвердить. Ошибка проверки ведёт к отказу без сохранения.	20
5 · Правила	start < end; та же лаборатория; start < bookedEnd И end > bookedStart. Сравниваем с активными записями.	20

6 · Тесты	Свободно: разрешить; пересечение: занято; соседние записи: разрешить; другая лаборатория: разрешить; обратный и нулевой интервалы: неверное время. Есть вывод.	15

> Учебный симулятор обрабатывает фиксированные сценарии последовательно. Он не является настоящим многопользовательским сервисом бронирования и не доказывает корректность конкурентных транзакций. Эта граница — хороший вопрос для сильных студентов.

9. Оценивание понимания

Предлагаемая оценка: 40% — прохождение этапов, 40% — индивидуальное объяснение, 20% — согласованность проекта и тестов. Формула: 0,4 × баллы квеста + объяснение из 40 + проверка проекта из 20. Скорость не оценивается.

- Объяснение: предметная область и границы — 10; связи — 10; последовательность проверок — 10; пересечения и граничные случаи — 10.
- Проект: согласованность модели и алгоритма — 10; осмысленный вывод по тестам — 10.
- Каждый критерий: 10 — самостоятельно и верно; 5 — частично или с помощью; 0 — не объяснил либо сохраняется ошибка.

В паре этапы общие, объяснение индивидуальное. Скачанный JSON можно изменить вручную: проверять серверный прогресс и устную защиту, а не доверять файлу как защищённому доказательству.

10. Подсказки и типичные ошибки

Ошибка	**Наводящий вопрос**
Процессор выбран сущностью	О чём программа бронирования должна хранить записи, а что лишь обеспечивает её работу?
Ремонт помещён внутрь программы	Кто физически выполняет это действие? Что реально может сделать система?
Связь студент — лаборатория без бронирования	Где хранить время и отмену, если студент приходит много раз?
Сохранение до проверки	Какие некорректные данные успеют попасть в систему?

Соседние интервалы считаются конфликтом	Используется ли лаборатория первой записью после её конца?
Сравниваются все лаборатории	Могут ли два человека заниматься одновременно в разных помещениях?

11. Резервный план и дифференциация

Если сеть недоступна — выдать постановку на бумаге, нарисовать три сущности, связи и блок-схему, вручную пройти шесть тестов. Если API недоступен, использовать самостоятельный режим без кода и JSON-отчёт; вход в групповой режим требует сети. При нехватке устройств работать парами.

Тем, кто отстаёт, дать сущности и попросить объяснить связи. Тем, кто завершил, предложить отменённые записи, часовые пояса или две одновременные заявки и спросить, какие дополнительные требования появятся. Не требовать реализации за оставшееся время.

12. Как использовать прежнюю подготовку

VM-конструктор и прежние материалы доступны в разделе «Виртуальные машины». Они дают наглядный образ лаборатории и сохраняются для самостоятельного изучения. На этом уроке основная цель — проектирование программы бронирования, а не настройка Ubuntu и Debian. Не смешивать баллы разных квестов: для них отдельные группы и локальные профили.

После урока можно предложить расширение: хранение бронирований в БД, API или интерфейс. Это следующий этап реализации, а не обязательная часть текущей концептуальной модели.